

Relational Model

The relational model, proposed by E. F. Codd in 1970, stores data in tables and links them through keys. It is still the foundation of most business systems, including this LMS.

Tables and Keys

Prerequisites: none.

A relational database stores data in tables, also called relations. Each row (tuple) is one record, such as one student, and each column (attribute) holds one property, such as the email address. Every column has a data type, such as INTEGER, VARCHAR or TIMESTAMP.

A primary key uniquely identifies each row and can never be NULL. A natural key comes from the data itself, like an email; a surrogate key is an artificial id, usually an auto-incrementing integer, and is the common choice because natural values can change.

A foreign key is a column that refers to the primary key of another table. In this LMS, the enrollments table has a student_id column that refers to users.id. Foreign keys enforce referential integrity: the database refuses an enrollment for a student who does not exist.

A candidate key is any column set that could serve as the primary key, and a composite key uses more than one column, for example (student_id, course_id) in an enrollments table. Constraints such as NOT NULL, UNIQUE and CHECK protect data quality at the database level.

ER Diagrams

Prerequisites: Tables and Keys.

An Entity-Relationship (ER) diagram is a visual design of a database before any table is created. Entities are the things we store, such as Student, Course and Teacher, drawn as rectangles. Attributes are their properties. Relationships connect entities, such as a Student "enrolls in" a Course.

Cardinality describes how many entities take part in a relationship. One-to-one: each user has one profile. One-to-many: one teacher owns many courses, but each course has one teacher. Many-to-many: a student enrolls in many courses and a course has many students.

When converting an ER diagram into tables, a one-to-many relationship becomes a foreign key on the "many" side, so courses gets a `teacher_id` column. A many-to-many relationship needs a separate junction table, also called an associative table, such as enrollments with `student_id` and `course_id`. The junction table can carry its own attributes, such as the enrollment date and progress.

Crow's foot notation is the most common way to draw cardinality in industry tools.

Normalization

Prerequisites: Tables and Keys, ER Diagrams.

Normalization organises tables to remove redundancy and the update anomalies it causes. If a teacher's phone number is copied into every row of a course table, changing it requires many updates and missing one creates inconsistent data.

First Normal Form (1NF): every column holds a single, atomic value, with no lists or repeating groups. Second Normal Form (2NF): the table is in 1NF and every non-key column depends on the whole primary key, not just part of a composite key. Third Normal Form (3NF): the table is in 2NF and no non-key column depends on another non-key column, which removes transitive dependencies. A popular summary: every non-key column must depend on "the key, the whole key, and nothing but the key".

Boyce-Codd Normal Form (BCNF) is a slightly stricter version of 3NF.

Normalization trades fewer anomalies for more joins at query time. Analytics systems sometimes denormalize on purpose, copying data to make reads faster, which is a deliberate trade-off rather than a mistake.